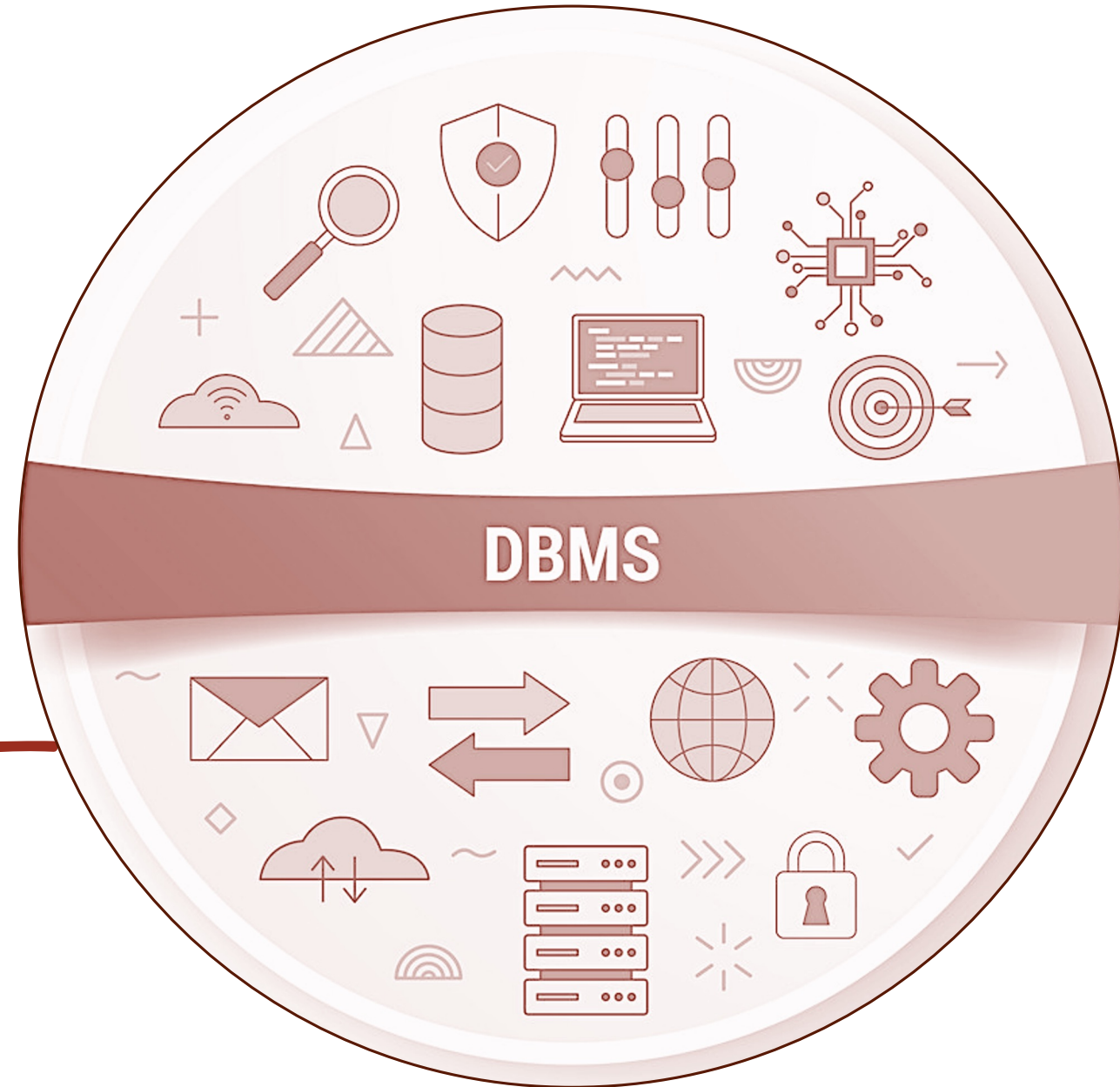


Database Management Systems Types



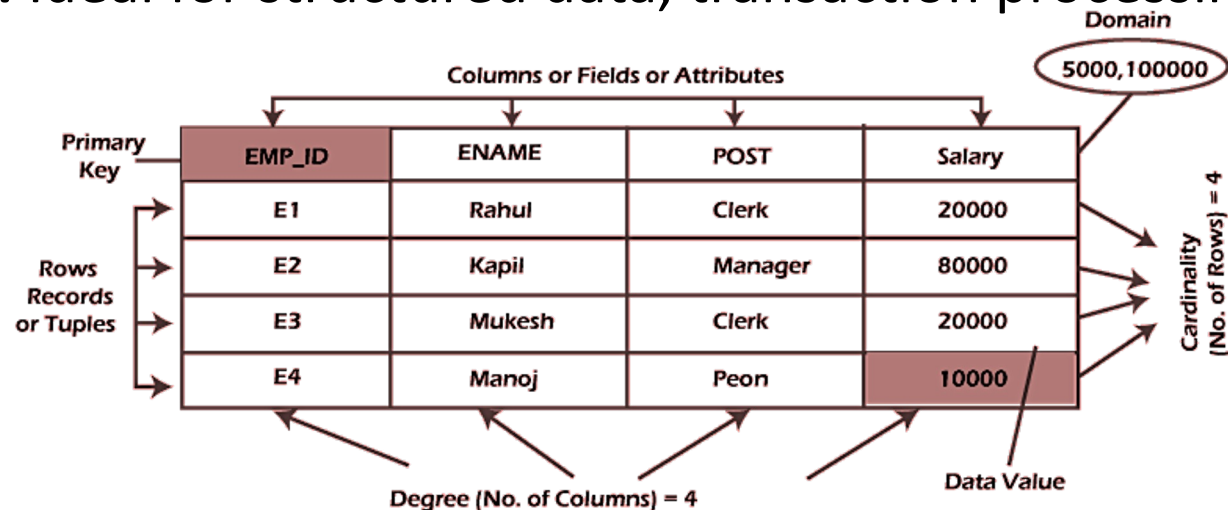


The Main Types Of DBMS You Will Discuss:

- Relational
 - Hierarchical
 - NoSQL
 - Object-Based
 - Object-Oriented
 - Distributed
- 

Relational database management system (RDBMS)

- stores data in structured tables with rows and columns. It enforces data integrity through relationships and constraints.
- **Key Features:** Tables, rows, columns, primary keys, foreign keys, and SQL for querying.
- **Examples:** MySQL, PostgreSQL, Oracle Database.
- **Use Cases:** Ideal for structured data, transaction processing, and reporting.

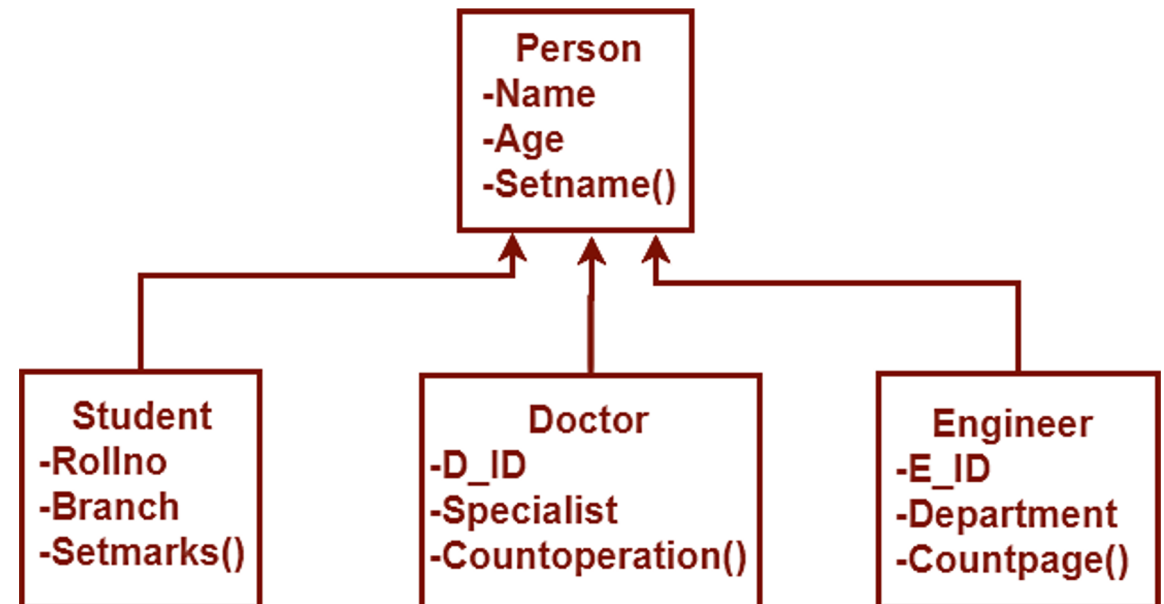


Object-Based DBMS

- **Definition and Characteristics:** An object-based DBMS extends the relational model to include objects with attributes and methods. It allows for more complex data modeling.
- **Key Features:** Objects, attributes, methods, encapsulation.
- **Examples:** db4o, ObjectDB.
- **Use Cases:** Object-oriented applications, complex data structures, scientific data.
- Object-based databases are the least commonly used among the listed DBMS types.
- They extend the relational model with objects, attributes, and methods, making them suitable for specific object-oriented applications.

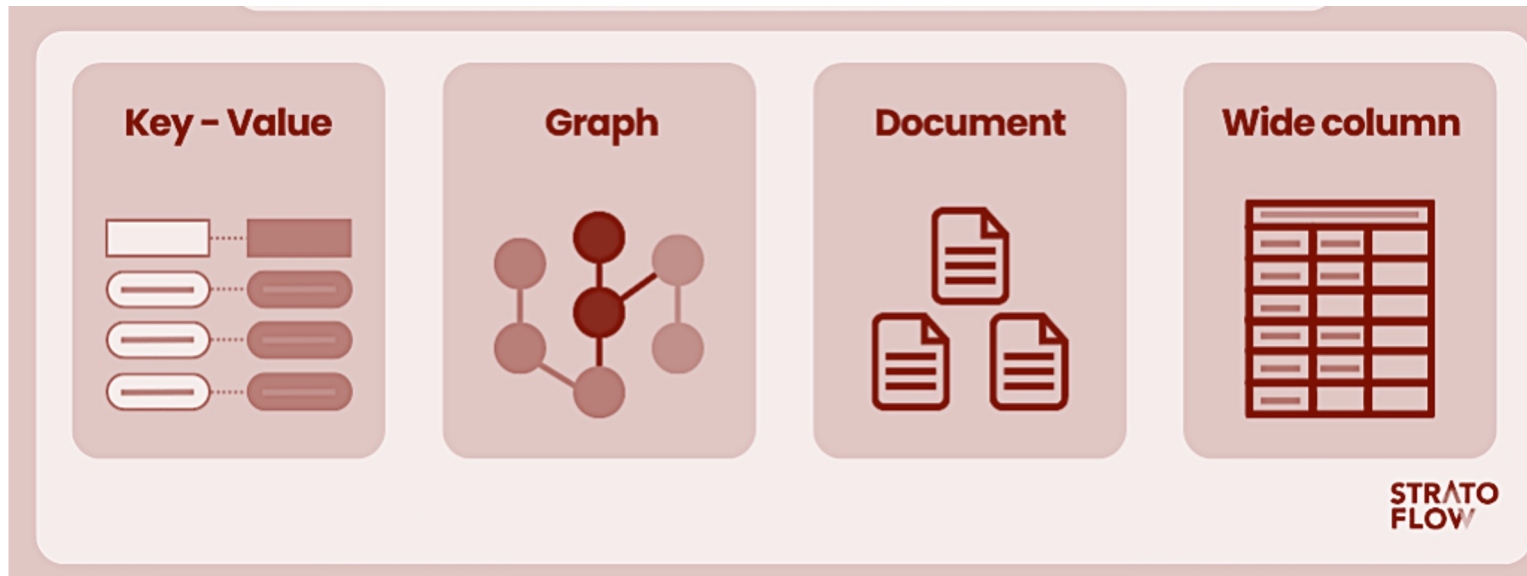
Object-Oriented DBMS

- **Definition and Characteristics:** An object-oriented DBMS (OODBMS) integrates objects directly into the database, allowing for inheritance and polymorphism.
- **Key Features:** Integration of objects and data, support for complex data modeling.
- **Examples:** ObjectStore, Versant.
- **Use Cases:** Multimedia databases, CAD systems, complex object-oriented applications.



NoSQL Databases

- NoSQL databases are designed to handle unstructured or semi-structured data and are often used in distributed, high-velocity environments.
- **Types of NoSQL Databases:**

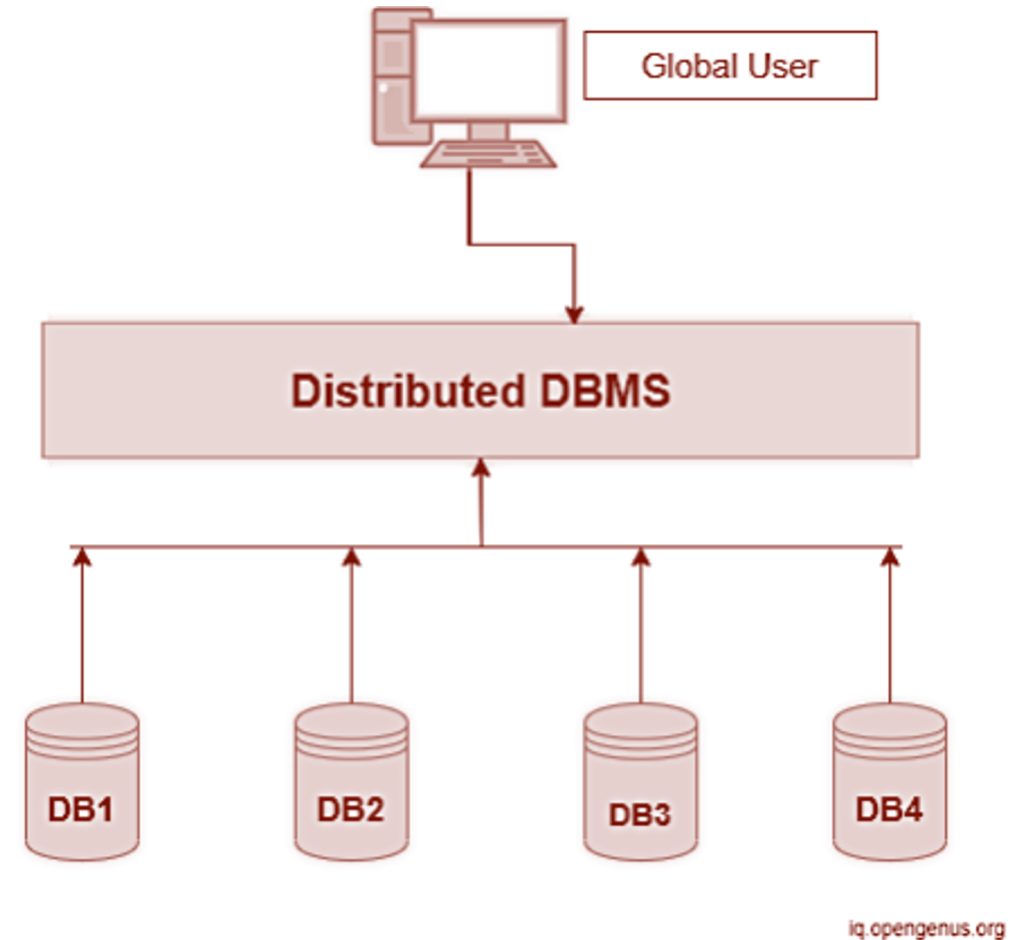


NoSQL Databases

- NoSQL databases are designed to handle unstructured or semi-structured data and are often used in distributed, high-velocity environments.
- **Types of NoSQL Databases:**
 - **Document-oriented** : Stores data in flexible, schema-less documents.
 - **Key-value**: Uses simple key-value pairs.
 - **Column-family** : Scales horizontally and stores data in column families.
 - **Graph** : Focuses on relationships between data entities.
- **Examples:** MongoDB (document), Redis (key-value), Cassandra (column-family), Neo4j (graph).
- **Use Cases:** Scalable web applications, real-time analytics, unstructured or semi-structured data.

Distributed DBMS

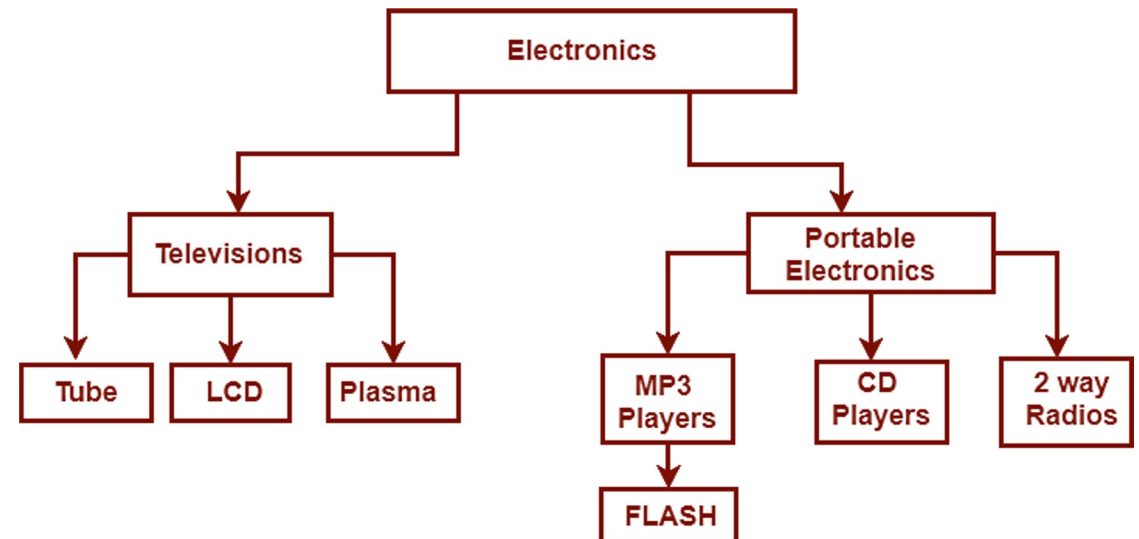
- spreads data across multiple locations or nodes. It offers scalability and fault tolerance.
- **Key Features:** Data distribution, multiple locations, replication, distributed transactions.
- **Examples:** Google Bigtable, Amazon DynamoDB, Bitcoin
- **Use Cases:** Scalable web applications, global data access, blockchain technology, fault tolerance.



Hierarchical database management system

- A hierarchical database organizes data in a tree-like structure with parent-child relationships. Each child can have only one parent.
- **Key Features:** Tree structure, parent-child relationships, rigid hierarchy.
- **Examples:** IBM's Information Management System (IMS).
- **Use Cases:** Legacy systems, data with a natural hierarchy, network infrastructures.

• Hierarchical databases are less common in modern systems but are still used in legacy environments.



Test your self

- Relational databases are widely used for managing _____ data.
- a) Structured
- b) Unstructured
- c) Hierarchical
- **In which type of DBMS are data represented using a tree-like structure with parent-child relationships?**
- a) Relational DBMS
- b) NoSQL DBMS
- c) Hierarchical DBMS
- d) Object-Based DBMS

Test your self

- **Which DBMS type is particularly suitable for managing unstructured or semi-structured data, as well as big data applications?**
 - a) Relational DBMS
 - b) Object-Oriented DBMS
 - c) NoSQL DBMS
 - d) Distributed DBMS
- **What is the primary advantage of a Distributed DBMS (DDBMS)?**
 - a) Simplicity in data modeling
 - b) Scalability and fault tolerance
 - c) Hierarchical data representation
 - d) Support for complex object-oriented data

